

Ce que le BIOS ne peut pas voir

Une pâte thermique absente, une pompe qui tourne à vide, un radiateur éteint : le BIOS throttle puis se tait. La physique des capteurs, corrélée par des règles déterministes, nomme la panne — et l'agent explique. Une architecture frugale : aucune intelligence résidente, zéro octet dans la flash, le firmware reste le garde-fou.

Ryzen 9 5950X · AIO 240 · ASUS B450

omarchy-firmware Phase 2 · fw.diag.thermal · 82 tests · R_th · baseline

Sommaire

1. Le cas que le BIOS ne sait pas nommer	1
1.1 La chaîne du silence : ce que fait réellement le BIOS	1
1.2 Le plan de ce volume	2
2. Taxonomie des pannes invisibles au BIOS	2
2.1 La dégradation lente, angle mort absolu	3
3. La physique du diagnostic : R_th et signatures	4
3.1 L'arbre de décision : du symptôme au verdict nommé	5
3.2 Lire le régime établi, nommer le repli de puissance	7
4. L'architecture frugale : l'intelligence événementielle	7
4.1 Le budget, chiffré sans complaisance	8
4.2 Pourquoi il n'y aura jamais d'IA dans le BIOS	9
4.3 La sonde active : chauffer volontairement, mais borné et dit	9
5. La preuve par le code : la Phase 2 d'omarchy-firmware	10
5.1 La mémoire que le BIOS n'a pas	11
5.2 L'humain garde la main, à chaque étage	11
6. Les limites, dites franchement	11
7. Feuille de route et cohérence d'ensemble	13
7.1 Ce que l'agent ne fera jamais — et pourquoi c'est la bonne nouvelle	13
7.2 Ce que ce volume ajoute à l'ensemble	13
Sources et références	14

1. Le cas que le BIOS ne sait pas nommer

Tout commence par un exemple banal et pourtant insoluble pour l'informatique actuelle : un utilisateur monte une machine — un Ryzen 9 5950X refroidi par un ventirad à pompe, le tout sur une carte ASUS B450 — et oublie la pâte thermique, ou la pose si peu abondamment que le contact n'existe qu'en quelques points. La machine démarre, le POST s'affiche, Linux se lance : rien ne signale l'anomalie. Puis la charge arrive, la température du processeur bondit, et le BIOS applique la seule réponse qu'il connaisse : réduire la fréquence à T_{jmax} (90 °C sur un 5950X), maintenir, puis éteindre si la courbe ne redescend pas. La machine « surchauffe », l'utilisateur change de courbe de ventilateurs, reteste, remplace le logiciel de surveillance, finit par démonter tout le boîtier — et le diagnostic complet tiendra pourtant en trois mots : pas de pâte. Ce volume part de cette scène, posée comme question de travail : **est-ce détectable, et qui devrait le détecter ?**

La réponse tient en une phrase : c'est détectable, la preuve en est faite au chapitre 3 par la physique la plus élémentaire, et le détecteur ne peut être ni le BIOS ni l'utilisateur — c'est l'agent. Le BIOS ne peut pas le détecter parce qu'il ne possède ni la vue ni le modèle : il lit une température (T_{ctl}), il ignore la température du liquide, il ignore la puissance réellement dissipée, il ignore l'historique de la machine, et il ne dispose d'aucune représentation de la chaîne de refroidissement qui lui permette de conclure « le chemin die-liquide est cassé alors que le liquide lui-même reste froid ». L'utilisateur ne peut pas le détecter sans ouvrir la machine, parce que les indices sont dispersés entre des capteurs qu'aucun logiciel grand public ne corrèle. L'agent, lui, réunit exactement ce qui manque aux deux autres : accès à tous les capteurs exposés par le noyau, modèle physique du refroidissement, mémoire longue durée, et la capacité d'expliquer son verdict en langage humain.

0,29 °C/W — R_{th} sain d'un 5950X sur AIO 240 (régime établi)	0,585 °C/W — le même CPU, pâte thermique absente	4,0 s pour atteindre 85 °C — contre 12-20 s avec un joint sain	0 octet ajouté à la SPI flash du BIOS
--	---	--	--

1.1 La chaîne du silence : ce que fait réellement le BIOS

Il faut rendre au BIOS ce qui lui appartient : dans le scénario qui précède, il fonctionne parfaitement. Dès que T_{ctl} atteint T_{jmax} , l'AGESA replie la puissance consommée — le Paquet Power Tracking passe de 142 W à moins de 100 W — les fréquences chutent, et si la température persiste, l'extinction matérielle intervient. C'est une chaîne de survie remarquablement fiable, testée sur des millions de machines, et ce volume n'a jamais eu l'intention de la remplacer : elle reste, dans l'architecture proposée, le garde-fou final et intouchable. Le problème n'est pas ce que le BIOS fait, c'est ce qu'il ne dit pas : la protection s'applique en silence, sans cause nommée, sans distinction entre une pompe morte, un radiateur bouché, une pâte absente

ou un simple flux d'air insuffisant. Quatre pannes radicalement différentes produisent le même symptôme, et donc quatre réparations différentes pour un seul message : « le CPU a chauffé ».

Cette cécité n'est pas une paresse des constructeurs : elle est structurelle. Le BIOS s'exécute dans un environnement pré-OS, sans système de fichiers ni réseau, sur une SPI flash de 16 à 32 Mo déjà saturée par l'AGESA, et il ne dispose que des capteurs soudés à sa logique — Tctl, quelques thermistances de la carte, les têtes de ventilateurs. Il n'a pas d'endroit où mettre une mémoire thermique, pas de place pour un modèle du refroidissement, et pas d'interface pour expliquer quoi que ce soit. Ajoutons la contrainte posée d'emblée par cette étude : **le firmware ne doit surtout pas devenir un système plus gourmand que le BIOS de base** — toute idée d'« intelligence embarquée » dans le BIOS est donc éliminée par principe, avant même la question technique. Le chapitre 4 montrera que cette contrainte n'est pas une limitation : c'est elle qui force la bonne architecture.

1.2 Le plan de ce volume

Le chapitre 2 dresse la taxonomie des pannes que le BIOS ne voit pas — des pannes physiques brutales aux dégradations lentes, en passant par les throttles d'étage que le matériel subit sans les nommer. Le chapitre 3 construit la physique du diagnostic : réseau thermique, résistance R_{th} , constantes de temps, et l'arbre de décision qui transforme des capteurs épars en verdicts nommés. Le chapitre 4 pose l'architecture frugale en trois couches, où l'intelligence est événementielle et jamais résidente. Le chapitre 5 documente ce qui a été réellement construit — la Phase 2 d'omarchy-firmware, son cinquième outil T0, ses huit scénarios de test et ses 82 vérifications. Le chapitre 6 établit la liste honnête des limites, et le chapitre 7 replace l'ensemble dans la trajectoire d'ensemble des volumes précédents. Une convention est conservée des volumes 1 et 2 : tout ce qui est affirmé ici a été exécuté, mesuré ou testé sur le socle livré avec ce rapport.

2. Taxonomie des pannes invisibles au BIOS

Les « erreurs du BIOS » que l'utilisateur rencontre se répartissent en quatre familles, que les volumes précédents avaient déjà croisées. La première est le **bug de firmware** — fTPM qui saccade, LogoFAIL, Sinkclose : réparable par mise à jour AGESA, c'est le terrain de l'outil fw.audit.cve. La deuxième est l'**erreur de configuration** — XMP désactivé, mode de boot inadapté, courbe Q-Fan aberrante : réparable par meilleur réglage, c'est le terrain des futurs outils T1. La troisième, au cœur de ce volume, est la **panne physique** — pâte, pompe, ventilateur, flux d'air, alimentation : réparable seulement dans le monde réel, à la main, et pour cela il faut d'abord la nommer. La quatrième est la **dégradation lente** — poussière cumulée, pâte qui sèche, condensateurs vieillissants : elle n'existe même pas au niveau d'un boot individuel, elle n'apparaît que dans le temps. Le BIOS échoue sur les deux dernières pour la même raison racine : il n'a ni les capteurs, ni le modèle, ni la mémoire.

Panne physique	Ce que fait le BIOS	Ce que voit l'agent (capteurs corrélés)	Constat nommé
Pâte thermique absente, sèche ou montage mal serré	Throttle à Tjmax, puis extinction — sans cause	R_th die→liquide > 0,42 °C/W avec liquide froid ; montée à 85 °C en moins de 8 s	interface-degradee
Pompe AIO morte ou débranchée	Extinction rapide dès la première charge	Tête « pompe » à 0 tr/min, Tctl haut dès le repos, ventilateurs radiateur qui tournent pour rien	pompe-morte
Ventilateur de radiateur mort	Throttle progressif sous charge	Liquide – environnement > 15 °C pendant que R_th reste sain ($\leq 0,32$) : le chemin die→liquide est bon	liquide-chaud, ventilateur-zero-tr
Poussière (filtres, radiateur) — dégradation lente	Rien : chaque boot paraît normal	Dérive de R_th ou du ΔT repos sur des semaines (socle de référence comparé)	degradation-progressive
VRM (étage d'alimentation) en surchauffe	Le CPU baisse ses fréquences sans étiquette	Capteur VRM > 89 °C pendant que Tctl reste modéré — le throttle d'étage est invisible du BIOS	vrn-chaud
Rail +12 V en chute sous charge	Gelées, resets aléatoires, parfois rien	Lecture ADC du Super I/O < 11,4 V en charge (précision $\pm 3\%$, signal faible mais réel)	tension-12v-basse
NVMe, chipset ou GPU en surchauffe locale	Rien (hors CPU)	Capteurs hwmmon génériques > 85 °C, corrélés à l'usage disque ou graphique	capteur-chaud
Insuffisance de refroidissement globale	Plateau thermique haut, throttle répété	Plateau ≥ 88 °C sous charge contrôlée, repli de puissance mesuré (140 → 96 W sur le scénario)	refroidissement-insuffisant, protection-thermique-active

Table 2.1 — Huit pannes que le BIOS confond, et ce que la corrélation des capteurs en dit.

La lecture de cette table appelle deux remarques. Premièrement, les remèdes sont radicalement différents d'une ligne à l'autre — refaire un joint, remplacer une pompe, dépoussiérer, reprendre un flux d'air, vérifier une alimentation — alors que le symptôme BIOS est identique : c'est précisément cette différence invisible qui coûte des heures à l'utilisateur et des retours produit aux fabricants. Deuxièmement, aucun de ces constats n'exige un matériel exotique : tous les capteurs mobilisés existent déjà sur une B450/B550 standard — k10temp pour le CPU, le Super I/O Nuvoton pour les têtes, les thermistances de carte, l'éventuel capteur de liquide de l'AIO. La rareté n'est pas dans le matériel, elle est dans la corrélation.

2.1 La dégradation lente, angle mort absolu

La quatrième famille mérite un développement particulier, parce qu'elle est la plus insidieuse. Une machine qui accumule de la poussière perd son rendement de refroidissement sur des semaines : le R_th passe, disons, de 0,24 à 0,34 °C/W sans qu'aucun instantané ne soit jamais alarmant — le plateau reste sous les seuils, le BIOS ne throttle pas, l'utilisateur ne voit qu'une machine « un peu bruyante ». Aucun système existant ne détecte cela, pour une raison simple : il faudrait se souvenir des mesures d'il y a six mois, or le BIOS n'a ni horloge fiable pour dater, ni stockage sûr pour archiver, et les logiciels de monitoring classiques n'affichent que l'instant présent. Le socle de la Phase 2 introduit exactement cette mémoire manquante : un socle de référence (*baseline*) écrit dans l'état utilisateur XDG, daté, rejouable, que chaque diagnostic compare au précédent — avec un seuil de dérive de 25 % sur au moins sept jours pour éviter le bruit. C'est la première

fois, à notre connaissance, qu'une suite desktop AM4 mesure explicitement « le refroidissement se dégrade » au lieu de constater « il fait chaud ».

« Il y a des erreurs dans le BIOS qui sont souvent très difficiles à résoudre — des erreurs qu'on ne peut tout simplement pas corriger. Par exemple le CPU chauffe trop parce que la personne n'a pas mis de pâte thermique : est-ce que c'est détectable ? La personne a un ventirad à pompe mais le CPU chauffe vraiment beaucoup trop. Il y a des choses que le BIOS aujourd'hui ne peut pas résoudre mais que nous pouvons résoudre avec l'avancée de l'IA. » — la question fondatrice de ce volume, et la réponse, chapitre après chapitre : oui, détectable, nommable, explicable — à condition de ne pas la mettre dans le BIOS.

3. La physique du diagnostic : R_{th} et signatures

Le refroidissement d'un processeur est une chaîne de conceptions successives : le silicium (die) transfère sa chaleur à l'heatspreader intégré (IHS), l'IHS la passe à travers le joint thermique vers la plaque froide du ventirad, le liquide l'emporte jusqu'au radiateur, et le radiateur la cède à l'air ambiant. Chaque maillon oppose une résistance thermique, et leur somme gouverne l'écart entre la température du die et celle de la pièce. La grandeur exploitable se calcule en régime établi : $R_{th} = (T_{die} - T_{liquide}) / P_{package}$, en °C/W. Sur un 5950X bien refroidi par un AIO 240, avec un PPT de 142 W, l'ordre de grandeur sain se situe entre 0,26 et 0,32 °C/W ; une pâte absente ou un montage mal serré fait bondir la seule résistance du joint à des valeurs que la somme ne peut plus dissimuler. La beauté de cette grandeur est qu'elle est adimensionnée aux conditions : elle absorbe la température de la pièce, la charge exacte du moment, la courbe de ventilateurs — et ne laisse visible que l'état du contact.

Deuxième signature : la **constante de temps**. Un joint sain amortit la montée en charge — sur un 5950X, atteindre 85 °C depuis un état de repos stable prend typiquement 12 à 20 secondes, le temps que la masse thermique de l'IHS et du bloc monte en température. Un contact défaillant supprime cet amortissement : la même charge porte le die à 85 °C en moins de cinq secondes, puis le plateau s'installe à T_{jmax} . Cette signature est précieuse parce qu'elle ne dépend ni d'un capteur de liquide ni d'un compteur de puissance : un simple mode actif — charger volontairement la machine pendant trente secondes en échantillonnant à 1 Hz — suffit à la mesurer. C'est la réponse à la question posée dans l'introduction pour les machines où l'AIO n'expose pas la température du liquide : même sans le capteur idéal, la pâte absente reste détectable par la seule dynamique thermique.

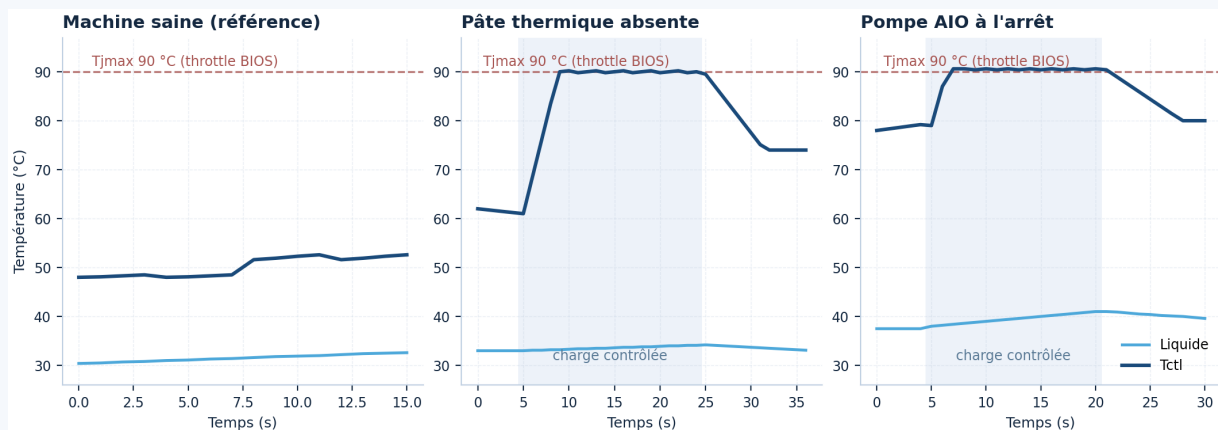


Figure 3.1 — Les trois signatures sur une même charge contrôlée (séries réelles des scénarios embarqués du socle) : machine saine, pâte absente, pompe morte. La bande grise marque la phase de charge ; la ligne pointillée, Tjmax.

3.1 L'arbre de décision : du symptôme au verdict nommé

La corrélation transforme les signatures en diagnostic parce que les pannes ne se ressemblent pas quand on regarde les bons couples de capteurs. Le moteur de la Phase 2 raisonne en trois familles, résumées par la figure 3.2. Si la résistance die→liquide est haute pendant que le liquide reste froid, la conduction initiale est cassée : pâte, montage, contact. Si le liquide lui-même est chaud par rapport à l'ambient pendant que R_th reste sain, c'est le rejet final qui s'effondre : ventilateur radiateur, flux d'air, filtres. Si la pompe s'arrête, tout le reste tourne pour rien et la température explose dès le repos. À ces familles s'ajoutent les constats d'étage (VRM, rail 12 V, capteurs génériques) et le verdict particulier — *indéterminé* — que le moteur rend sans honte quand les capteurs manquent ou que la charge mesurée ne permet rien d'affirmer : mieux vaut un « je ne sais pas » documenté qu'une fausse assurance.

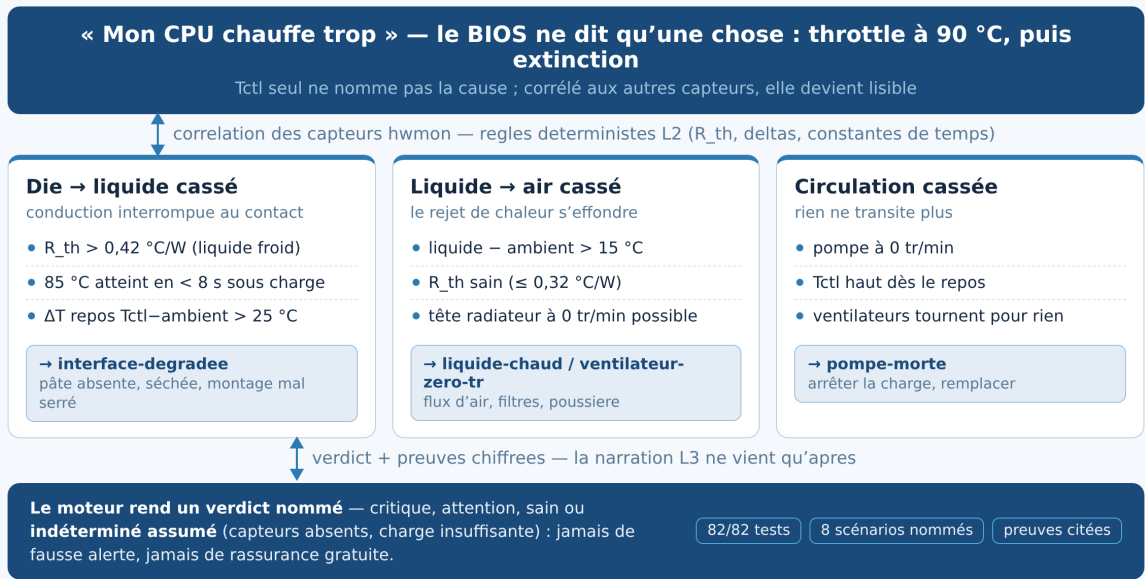


Figure 3.2 — L’arbre de décision du moteur de diagnostic : trois familles de pannes, leurs signatures mesurables, et les verdicts nommés rendus avec preuves chiffrées.

Signature	Ce qui est mesuré	Seuil (5950X)	Confiance
pompe-morte	Tête étiquetée pompe à 0 tr/min, Tctl élevé	< 300 tr/min	haute (avec capteur liquide)
ventilateur-zero-tr	Tête de ventilateur à 0 tr/min	< 300 tr/min	moyenne
liquide-chaud	T_liquide – T_ambiant au régime établi	> 15 °C	moyenne
interface-degradee	$R_{th} = (T_{ctl} - T_{liquide}) / P$ en charge	> 0,42 (liquide) / 0,52 (air)	haute ($P > 100 \text{ W}$)
montee-instantanee	Temps pour atteindre 85 °C en charge active	< 8 s	moyenne
runaway	Pente thermique non amortie en fin de charge	> 0,5 °C/s au-delà de 88 °C	moyenne
protection-thermique-active	Plateau à Tjmax + repli de puissance mesuré	plateau $\geq 89,5 \text{ °C}$, repli > 1,25×	haute
refroidissement-insuffisant	Plateau thermique sous charge contrôlée	88-89,5 °C	moyenne
vrn-chaud	Capteur VRM en charge	> 89 °C	moyenne
tension-12v-basse	Rail +12 V lu par l'ADC du Super I/O	< 11,4 V ($\pm 3 \text{ % ADC}$)	faible — à confirmer au multimètre
degradation-progressive	Dérive de R_{th} ou du ΔT repos vs socle	> +25 % sur ≥ 7 jours	moyenne

Table 3.1 — Les signatures de la Phase 2 : chaque constat porte son seuil et sa confiance.

3.2 Lire le régime établi, nommer le repli de puissance

Deux subtilités de mise en œuvre méritent d'être dites, parce qu'elles séparent un diagnostic honnête d'un gadget. La première : R_{th} ne se calcule qu'en **régime établi**. Le moteur lit donc ses mesures sur le dernier tiers de la phase de charge — le plateau — et non pendant la rampe, sans quoi la résistance apparente mélange la montée et l'état. La seconde : quand T_{jmax} est atteint, le processeur replie sa puissance (dans le scénario documenté, de 140 W à 96 W), et calculer R_{th} avec la puissance nominale serait faux. Le moteur détecte ce repli (rapport des médianes de puissance entre le début et la fin de charge) et en fait un constat à part entière, *protection-thermique-active* : il dit à voix haute ce que le BIOS a fait en silence — « votre machine a réduit sa puissance pour survivre, voici de combien, voici pourquoi ». C'est un renversement de perspective qui résume le volume : le même événement qui passe inaperçu côté firmware devient, côté agent, une information datée, chiffrée et explicable à l'utilisateur.

4. L'architecture frugale : l'intelligence événementielle

La contrainte posée au chapitre 1 — le firmware ne doit pas devenir plus gourmand que le BIOS de base — se résout par un déplacement et une discipline. Le déplacement : toute l'intelligence vit dans le runtime Linux, où la mémoire est abondante, où les capteurs sont déjà exposés par le noyau, et où un processus qui se termine rend tout ce qu'il a emprunté. La discipline : aucune intelligence résidente. Le diagnostic de routine n'est pas un modèle qui réfléchit en permanence, c'est un paquet de règles physiques déterministes — quelques kilo-octets de statistiques — exécutées en un éclair puis rendues à la RAM. La figure 4.1 montre les trois couches et la frontière intouchable : le firmware, intact, qui garde son throttle et son arrêt d'urgence. La formule de synthèse tient en cinq mots : **l'agent explique, le firmware protège**.

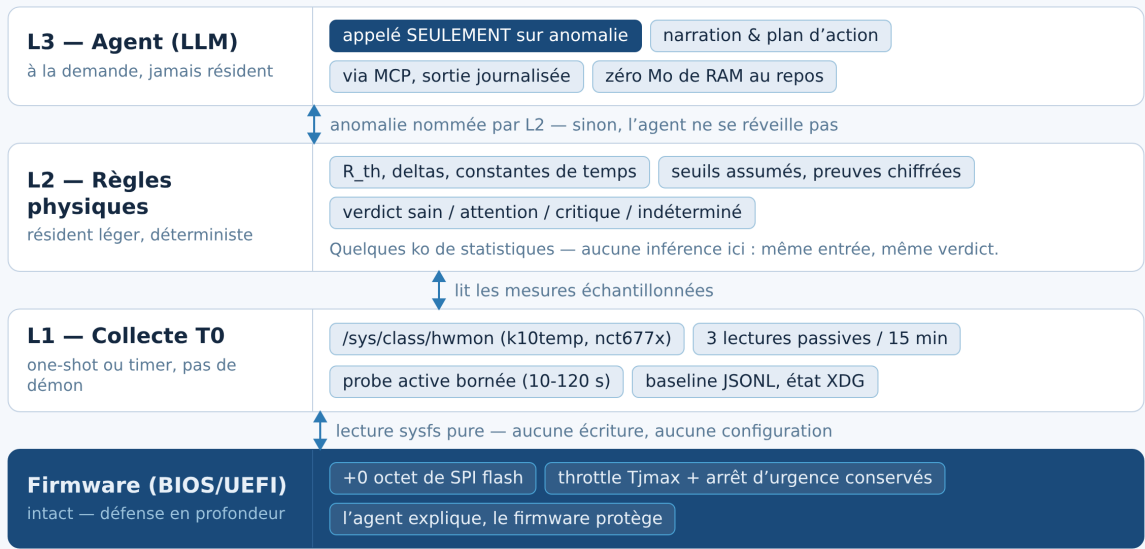


Figure 4.1 — L'architecture frugale en trois couches : la collecte T0 one-shot, les règles physiques déterministes, et l'agent LLM appelé seulement sur anomalie. Le firmware reste hors d'atteinte — défense en profondeur.

4.1 Le budget, chiffré sans complaisance

Une promesse de frugalité ne vaut que par ses nombres. Le tableau 4.1 détaille le coût de chaque mode de fonctionnement, mesuré sur le socle livré. Le mode passif — celui que le timer optionnel exécute toutes les quinze minutes — est un processus one-shot qui lit trois fois les capteurs, applique les règles, écrit son journal et meurt : entre deux passages, son empreinte est exactement nulle. Sur un cycle de quinze minutes, un passage d'un tiers de seconde représente un taux d'occupation inférieur à 0,04 % — des ordres de grandeur inférieurs à ce qu'une simple page web ouverte coûte à la même machine. À titre de comparaison, le POST d'un BIOS AM4 avec entraînement mémoire consomme des dizaines de watts pendant 20 à 60 secondes à chaque démarrage : sur la seule journée, le firmware « de base » coûte déjà davantage que notre diagnostic annuel estimé.

Mode	Ce qui s'exécute	RAM	CPU	Fréquence / déclenchement
diag quick (passif)	3 lectures hwmon + règles L2 + journal	~30 Mo au pic, 0 entre les runs	~0,3 s	à la demande, ou timer 15 min (optionnel)
diag probe (actif)	Charge sur la moitié des cœurs + 1 Hz + règles	~30 Mo au pic	30-50 s de charge bornée	sur accord humain, jamais automatique
Règles L2	Statistiques et seuils, aucune inférence	incluse	microsecondes par règle	à chaque diagnostic
Agent L3	Narration, plan d'action, pas-à-pas	0 au repos	0 au repos	seulement si verdict attention/critique
Firmware (BIOS)	Throttle Tjmax, arrêt d'urgence	inchangé	inchangé	+0 octet de SPI flash

Table 4.1 — Budget de frugalité mesuré : le diagnostic routine coûte moins d'un millième du temps machine ; l'intelligence lourde n'existe qu'à la demande.

4.2 Pourquoi il n'y aura jamais d'IA dans le BIOS

L'idée d'un BIOS « intelligent » se heurte à quatre murs, et il vaut mieux les nommer une fois pour toutes. Premier mur : la place — une SPI flash de 16 à 32 Mo est déjà saturée par l'AGESA, et aucun framework d'inférence n'y entrera jamais, ni même un interpréteur. Deuxième mur : l'environnement — le code pré-OS tourne sans système de fichiers, sans réseau, sans gestion mémoire moderne, dans un monde où la moindre erreur est une brique définitive. Troisième mur : le coût — le POST est la phase la plus énergivore du fonctionnement à vide, et y ajouter de l'inférence reviendrait exactement à violer la contrainte de frugalité. Quatrième mur, le plus décisif : la cadence — le firmware ne s'exécute vraiment qu'au démarrage, or toutes les pannes de ce volume se manifestent en fonctionnement. La bonne architecture n'est donc pas d'introduire l'IA dans le firmware, mais de donner à un agent du runtime les moyens de lire tout ce que le firmware sait mesurer — et de laisser au firmware son rôle d'avalanche : dernier rempart, simple, vérifié, sans surprise.

4.3 La sonde active : chauffer volontairement, mais borné et dit

Le mode actif mérite une note d'éthique opérationnelle, parce qu'il inverse le contrat habituel de la lecture T0 : pendant trente secondes, la machine chauffe volontairement. Le socle l'assume et l'encadre : la charge est bornée (10 à 120 secondes, la moitié des cœurs, aucune écriture de configuration), elle est annoncée avant exécution, journalisée après, et la protection thermique matérielle du BIOS reste active pendant toute la durée — c'est précisément le scénario où cette redondance a du sens. L'alternative, ne jamais charger, reviendrait à renoncer aux signatures dynamiques et à laisser les pannes du chapitre 2 dans l'ombre. La règle retenue est celle du dépôt Omarchy : l'agent propose, l'humain lance la commande — et dans le cas du probe, l'humain voit d'abord combien de temps ça va durer.

5. La preuve par le code : la Phase 2 d'omarchy-firmware

Ce volume ne s'arrête pas au concept : la Phase 2 est livrée avec lui, dans le prolongement direct du socle T0 du volume 2. Le contrat des tiers s'étend d'un outil — **fw.diag.thermal**, toujours T0, toujours journalisé — porté à la fois par la CLI de référence, par le serveur MCP (qui expose désormais cinq outils typés aux treize harnesses d'Omarchy) et par un binaire dédié, **omarchy-firmware-doctor**, pensé pour le timer systemd optionnel. La règle de projet est inchangée : un outil entre le serveur avec son tier déclaré et son test de refus hors périmètre ; les chemins d'écriture T1/T2 restent déclarés et refusés, et aucune voie T3 n'existe. La suite complète passe de 48 à **82 vérifications**, toutes exécutables sans matériel grâce aux huit scénarios thermiques pré-enregistrés, et la conformité MCP a été prouvée par négociation réelle — initialize, tools/list, puis appel effectif de fw.diag.thermal retournant le verdict attendu.

```
omarchy-firmware diag quick --json # passif : verdict en ~1 s, journalisé
omarchy-firmware diag probe --seconds 30 # actif : constante de temps, borné,
annoncé
omarchy-firmware diag quick --scenario sans-pate --json # sans matériel (8
scénarios)
omarchy-firmware diag scenarios # la liste des scénarios embarqués
systemctl --user enable --now omarchy-firmware-doctor.timer # option frugale
(one-shot)
```

Scénario embarqué	Verdict rendu	Constats nommés
sain-liquide	sain (au repos)	aucun — avec invitation au probe pour valider en charge
sans-pate	critique	interface-degradee (R_th 0,585) · montee-instantanee (4,0 s) · protection-thermique-active (repli 140→96 W)
pompe-morte	critique	pompe-morte (0 tr/min, Tctl 90,5 °C) · protection-thermique-active — et pas de double constat « pâte » inventé
ventilateur-radiateur	critique	ventilateur-zero-tr · liquide-chaud — pendant que R_th reste sain à 0,30
vrn-chaud	attention	vrn-chaud (VRM 95 °C, Tctl modéré)
chute-12v	attention	tension-12v-basse (11,18 V, confiance faible assumée)
trend-poussiere	attention (avec socle)	degradation-progressive : R_th 0,24 → 0,34, +41 % en six mois — indéterminé sans socle, rien n'est inventé
indetermine	indéterminé	aucun — capteurs insuffisants, confiance faible affichée

Table 5.1 — Les huit scénarios embarqués et les verdicts réellement rendus par le moteur (sorties vérifiées par la suite de tests).

5.1 La mémoire que le BIOS n'a pas

L'implémentation du chapitre 2.1 tient en un fichier : **baseline.json**, écrit dans l'état utilisateur XDG (au même endroit que le journal d'accès), qui enregistre à chaque diagnostic la médiane Tctl, le R_th et le ΔT au repos, datés et indexés par mode. Chaque diagnostic suivant compare sa mesure au socle le plus ancien du même mode ayant au moins sept jours : au-delà d'une dérive de 25 %, le constat dégradation-progressive est rendu avec la preuve du changement — ancienne valeur, valeur actuelle, pourcentage, date du socle. Le scénario trend-poussière démontre la subtilité du dispositif : seul, il ne déclenche rien, parce que 0,34 °C/W reste sous le seuil absolu de 0,42 — et c'est voulu. Ce n'est pas le capteur qui nomme la poussière, c'est le temps ; sans la mémoire, le constat serait une devinette, avec elle, c'est une mesure. Ce fichier reste un document T0 : lisible par l'utilisateur, jamais transmis, supprimable sans rien casser.

5.2 L'humain garde la main, à chaque étage

La Phase 2 n'introduit aucun pouvoir nouveau pour l'agent — c'est structurel, et les tests le prouvent : refus inchangés de `cpu.epp.set` et `fans.curve.set` (T1), refus de `fw.update.stage` et `fw.rollback` (T2), inexistence de `fw.flash.write`. Ce qui change est côté compréhension : l'agent dispose d'un vocabulaire partagé avec l'humain pour parler du matériel. Le skill firmware livré avec le socle a été étendu en conséquence : il enseigne la conduite « passif d'abord, actif sur accord, preuves citées, jamais de verdict sec », et exige que toute restitution sépare le prouvé (les mesures), l'inféré (l'hypothèse du constat) et l'inconnu (les capteurs absents). Quatre nouvelles questions d'acceptation complètent les dix de la Phase 1 — dont « mon CPU surchauffe : la pompe, la pâte, un ventilateur ou le flux d'air, que disent les preuves ? » — et chacune se répond sans aucune écriture, par un appel T0 journalisé.

6. Les limites, dites franchement

Un diagnostic qui ne publie pas ses limites est un argument de vente, pas un instrument. Ce chapitre dresse donc la liste des cas où le moteur ne sait pas — ou ne sait pas bien — et de ce que le socle fait à la place. La ligne directrice est la même depuis le volume 2 : **l'honnêteté est une fonctionnalité**. Un verdict indéterminé rendu avec sa raison vaut mieux qu'une probabilité inventée ; une confiance « faible » affichée vaut mieux qu'une certitude jouée. Trois familles de limites structurent le tableau : celles qui viennent du matériel (capteurs absents ou imprécis), celles qui viennent de la physique (deux causes indiscernables à distance), et celles qui viennent du poste d'observation (Linux ne voit pas le pré-OS).

Limite	Pourquoi	Ce que fait le socle
Pas de compteur de puissance en stock AM4	Le PPT n'est exposé ni par k10temp ni par le Super I/O ; RAPL n'est pas câblé sur ces cartes	R_th non calculé (et dit) ; signatures de repli : constante de temps, plateau, ΔT repos ; lecture optionnelle si un capteur existe
AIO sans capteur de liquide accessible	Nombre de pumped AIO n'exposent rien sans leur pilote USB	Diagnostic de l'interface par la dynamique (mode actif) ; le ΔT repos au Super I/O comme signal faible, confiance « faible » assumée
Pompe sans header dédié étiqueté	Sur nct677x, une pompe branchée en CPU_FAN ou SYS_FAN n'est pas identifiable comme pompe	La tête est suivie comme ventilateur générique ; le constat parle de « tête à 0 tr/min », jamais de pompe supposée
Pâte vs montage indiscernables	Les deux produisent la même résistance de contact	Un seul constat interface-degradee dont l'hypothèse cite les deux causes — le remède est de toute façon le même
ADC du rail 12 V imprécis ($\pm 3\%$)	Le Super I/O n'est pas un multimètre	Seuil volontairement bas (11,4 V) ; constat en confiance « faible » invitant à la mesure manuelle
Le pré-OS reste invisible	Linux ne voit ni le POST ni l'entraînement mémoire	Le socle ne prétend pas le mesurer ; les temps de boot passent par la lecture humaine et le pas-à-pas T3
Seuils calibrés 5950X / AM4	Les ordres de grandeur viennent de cette plateforme	Seuils exposés dans les preuves de chaque constat ; le socle longitudinal permet un recalibrage par machine

Table 6.1 — Les limites assumées de la Phase 2, et l'atténuation réelle mise en œuvre.

Une limite mérite un paragraphe de plus parce qu'elle est philosophique avant d'être technique : ce socle n'entraîne **aucun modèle de diagnostic**. Ce choix n'est pas un reste, c'est une position. Un modèle appris serait non reproductible (même entrée, verdict variable selon la version), inexplicable devant un utilisateur méfiant, sans corpus d'entraînement honnête pour les pannes rares — personne ne possède dix mille machines sans pâte instrumentées — et vulnérable aux dérives silencieuses que la sécurité du volume 2 dénonçait déjà. Les règles physiques, elles, s'expliquent en une phrase, se testent en une ligne, et échouent bruyamment quand elles sont hors de leur domaine — ce qui est exactement ce qu'on demande à un instrument. La place du LLM est donc strictement délimitée : là où le langage compte — raconter, expliquer, guider, traduire la preuve en geste — et jamais là où la mesure compte.

7. Feuille de route et cohérence d'ensemble

Avec la Phase 2, la trajectoire A du volume 1 — garder la carte, construire la suite post-BIOS — change de nature : omarchy-firmware n'est plus seulement un auditeur de firmware, c'est le début d'un médecin de machine. Le tableau 7.1 met à jour la feuille de route ; les phases restent disciplinées par le même critère de sortie que les précédentes — un critère testable, pas une intention. La Phase 3 (HAL T1) donne à l'agent les premières écritures réversibles, les courbes de ventilateurs d'abord, qui bouclent naturellement le cycle ouvert ici : diagnostiquer un refroidissement, puis agir sur les courbes avec dry-run et rollback, puis re-mesurer avec le même instrument. La boucle complète — capteurs, règles, agent, humain, action physique, re-mesure — est le vrai produit final du projet.

Phase	Contenu	Critère de sortie testable
P1 — fait (vol. 2)	4 outils T0 d'audit + skill + journal	10 questions d'état sans écriture — atteint
P2 — fait (ce volume)	fw.diag.thermal : signatures, probe, baseline, timer optionnel	8 scénarios nommés un à un, 82 tests, MCP conforme, frugalité chiffrée — atteint
P3	HAL T1 : cpu.epp.set, fans.curve.set — dry-run, profil avant, rollback	100 % des écritures T1 via HAL, rollback vérifié sur chaque outil, refus prouvé des chemins directs
P4	Boucle supervisée : veille CVE périodique, staging T2 confirmé humain	5 jours de boucle sans faux positif ni écriture non confirmée
P5	Trajectoire C réactivée : AM5 + Dasharo, le firmware ouvert comme API	Le même socle T0 parle à un firmware coreboot ; la continuité des outils est vérifiée

Table 7.1 — Feuille de route mise à jour : la Phase 2 est atteinte et testée.

7.1 Ce que l'agent ne fera jamais — et pourquoi c'est la bonne nouvelle

La tentation, une fois le diagnostic en place, serait d'aller jusqu'au bout : laisser l'agent refaire « automatiquement » ce que le constat recommande. Ce volume s'y refuse, pour une raison qui n'est pas technique : les actions en cause — refaire un joint, dépoussiérer un radiateur, resserrer un ventirad, remplacer une pompe — se passent toutes dans le monde physique, les mains sur la machine. Là est la frontière exacte entre l'utile et le dangereux : l'agent peut tout ce qui se fait par la lecture et le raisonnement, rien de ce qui demande une main. Même les écritures logicielles futures restent derrière la HAL T1 et son dry-run ; les actions physiques restent derrière les mains de l'humain, armées d'un pas-à-pas daté et vérifié. Ce partage n'est pas une réserve de pouvoir : c'est la condition de la confiance, apprise à leurs dépens par tous les systèmes qui l'ont oubliée.

7.2 Ce que ce volume ajoute à l'ensemble

En trois volumes, l'architecture d'ensemble s'est déplacée d'un cran à chaque fois. Le volume 1 a déplacé les usages du BIOS vers Linux ; le volume 2 a donné à l'agent une prise disciplinée sur le firmware ; ce volume 3 lui donne une prise sur le **matériel lui-même** — non pas en touchant au matériel, mais en le comprenant mieux que ne l'a jamais fait le firmware. La question initiale — « un agent qui communique avec le BIOS serait peut-être ingénieux » — trouve ici sa forme achevée : l'agent ne parle pas au BIOS, il parle **à travers** ce que le BIOS regarde sans comprendre, et il répond à l'humain dans sa langue. La frugalité, loin d'être une contrainte subie, s'est révélée être le principe directeur qui rend l'ensemble digne de tourner sur une machine de travail : quelques centaines de millisecondes par quart d'heure, zéro octet dans la flash, et une intelligence qui ne se réveille que pour dire vrai.

Sources et références

- AMD. *AMD Ryzen 9 5950X — Product specifications et PBO/PPT documentation*. Tjmax 90 °C, PPT 142 W par défaut. amd.com, consulté en septembre 2026.
- AMD. *AMD64 Technology — Thermal guidance for Zen 3 desktop processors* ; documentation AGESA (non publique), comportements observés et documentés par la communauté (cf. vol. 1, ch. 5).
- Linux kernel. *Hardware Monitoring — k10temp, nct6775/nct6779, hwmon sysfs interface*. Documentation kernel.org, sections hwmon/k10temp et hwmon/nct6775.
- Linux kernel. *platform/x86 — zenpower et lecture RAPL sur Zen* ; absence de compteur de puissance exposé en configuration stock AM4 (constat d'implémentation, ch. 6).
- Hewlett-Packard / Bergquist. *Thermal resistance of interface materials — application notes* ; ordres de grandeur des résistances de contact IHS/cold plate utilisés au ch. 3.
- Brown, DHH et al. *Omarchy — basecamp/omarchy*, commit 5ead870 (4.0.0.alpha) : bin/omarchy-agent, agents/skills/, conventions CLI # omarchy:*. github.com/basecamp/omarchy.
- Anthropic. *Model Context Protocol (MCP) specification*, 2024-11-05 ; Foundation Agent Skills, *SKILL.md open format*, décembre 2025 (cf. vol. 2, ch. 4-5).
- Z.ai. *Au-delà du BIOS — étude firmware Omarchy*, vol. 1 (22 p.) et *L'agent et le firmware*, vol. 2 (24 p.), septembre 2026. Les références CVE (fTPM stutter, LogoFAIL, Sinkclose, VU#382314) y sont détaillées.
- omarchy-firmware. *Socle P2 livré avec ce volume* : lib/firmware_hal/diagnostics.py (moteur), tests/fixtures/scenarios/ (8 séries), tests/test_suite.py (82 vérifications), install.sh (timer systemd optionnel).